

Worksheet No. - 4

Student Name: Anjali kumari

Branch: MCA general

Semester: 3rd

Subject Name: Technical Training-II.

Subject Code: CONT_25CAH-707

UID: 25MCA20023

Section: Group:1/A

Date of Performance: 8 Aug.

Experiment 4 : Backend Development for Job Portal with Authentication and Database Integration.

1. Aim:

A company is developing a job portal where users can register, create profiles, and apply for jobs. The backend system must handle authentication, job listings, and user data securely.

2. Objective:

- Develop a web-based Student Management System using Node.js And MongoDB.
- Implement user registration and login authentication.
- Store and manage user credentials using `users.txt`.
- Display employee personal, contact, and job information dynamically.
- Display salary details including Basic Salary, HRA, Medical Allowance, Bonus, and Annual Salary.
- Provide salary visualization using charts and a live current-time display.
- Create a responsive and user-friendly dashboard using HTML, CSS, Bootstrap, and JavaScript.

3. Task to be done :

1. Create the Node.js server and project folder structure.
2. Design the **Login and Registration** pages.
3. Implement registration and login routes with credential validation.
4. Create and read employee data from `employee.txt`.
5. Develop the dynamic **Employee Dashboard**.
6. Add salary charts, profile image, logo, and live time functionality.
7. Test the complete system and fix routing, file-loading, and authentication errors.

4. Online Resources:

- Node.js Official Documentation
- EJS Official Documentation
- MDN Web Docs (HTML, CSS, JavaScript)
- W3Schools – HTML, CSS, JavaScript
- GeeksforGeeks – Node.js and Express.js Tutorials

Software Requirements

- Node.js (Latest LTS Version)
- Visual Studio Code
- Google Chrome / Microsoft Edge / Mozilla Firefox
- Command Prompt / Windows PowerShell / Terminal
- npm (Node Package Manager)

Standard Tools Used

- Node.js
- HTML5
- CSS3
- JavaScript
- File System (fs) Module
- Browser (Chrome/Edge/Firefox) for user input and displaying the output

5. Procedure/ Algorithm:

```
<section class="hero">
```

```
<div class="container">
```

```
<!-- LOGO -->
```

```

```

```
<!-- ANIMATED TITLE -->
```

```
<h1 class="welcome-title">
```

```
  Employee Salary Management System
```

```
</h1>
```



UNIVERSITY INSTITUTE of
COMPUTING
Asia's Fastest Growing University



<!-- ANIMATED SUBTITLE -->

<p class="welcome-subtitle">

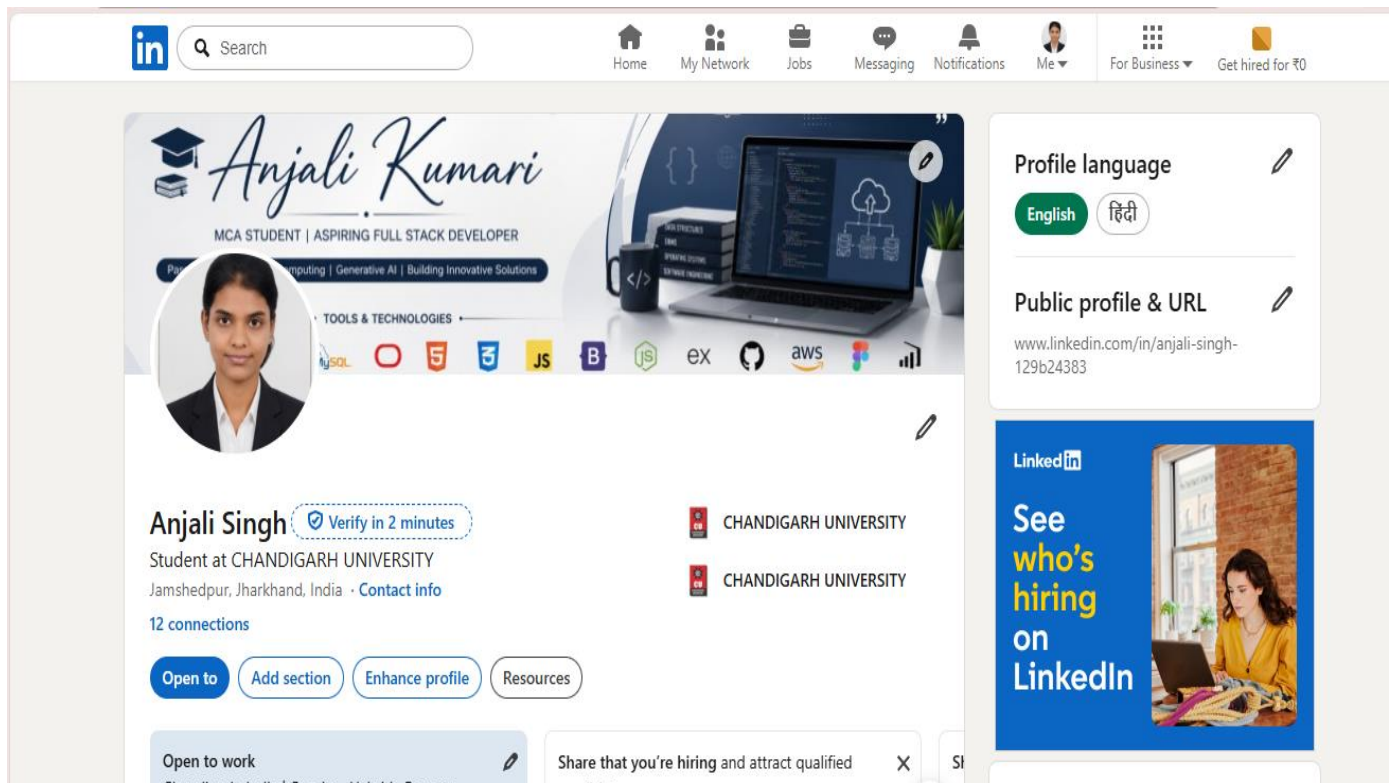
Secure Employee Management & Salary Dashboard

</p>

</div>

</section

5 OUTPUT:





Search & Filter

Search by name, roll number, email or course...

Search

Show All

All Courses

All Semesters

All Gender

Admission Year

Filter

Student Records

5 Records

Name	Roll Number	Email	Phone	Course	Semester	Age	Gender	Admission Year	Actions
Anjali Kumari	25MCA20023	anjali23@example.com	9876543210	MCA	2	23	Female	2025	<button>Edit</button> <button>Delete</button>
Rahul Sharma	25MCA20024	rahul24@example.com	9876543211	MCA	2	24	Male	2025	<button>Edit</button> <button>Delete</button>
Priya Singh	25BCA20025	priya25@example.com	9876543212	BCA	4	22	Female	2024	<button>Edit</button> <button>Delete</button>
Arjun Verma	25MCA20026	arjun26@example.com	9876543213	MCA	3	23	Male	2025	<button>Edit</button> <button>Delete</button>
Neha Gupta	25BCA20027	neha27@example.com	9876543214	BCA	2	21	Female	2025	<button>Edit</button> <button>Delete</button>

ORGANIZATION

Anjali's Org - 2026-0...

PROJECT

Project 0

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

- admin
- local
- sample_mflix
- studentDB

students

+

Cluster0 > studentDB > students

Documents 5

Aggregations

Schema

Indexes 3

Validation

🕒

Type a query: { field: 'value' } or [Generate query](#)

Expl

+

📄

</>

25

1 - 5 of

```
_id: ObjectId('6aa6c3a6720cbd76c224dfb3')
name: "Anjali Kumari"
rollNumber: "25MCA20023"
email: "anjali23@example.com"
phone: "9876543210"
course: "MCA"
semester: 2
age: 23
gender: "Female"
```

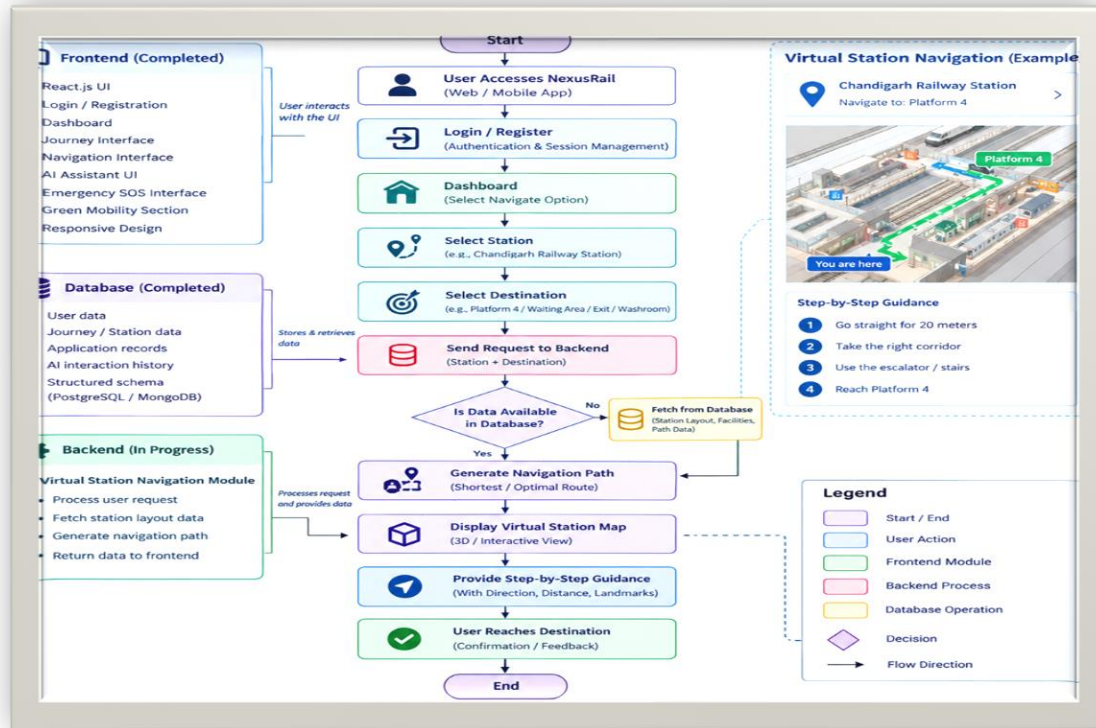


FIG:Flowchart

7.LEARNING OUTCOMES:

- Understand Mongoose and its role in connecting Node.js applications with MongoDB.
- Create MongoDB schemas and models using Mongoose.
- Perform CRUD operations — Create, Read, Update, and Delete student records.
- Connect an Express.js application with a MongoDB database using Mongoose.
- Develop RESTful APIs for managing student data.
- Handle database errors and validation using Mongoose.
- Test and verify APIs using tools such as Postman.
- Manage student records efficiently through a database-driven application.